

Aegis Architecture Specification (v0.3)

1. System Definition

Aegis is a deterministic automation orchestration framework that executes policy-validated actions under a supervised control loop. All modules operate through explicit data contracts and structured logging. No autonomous execution occurs without policy approval.

2. Module Inventory

Module	File	Primary Class	Responsibility
Control Loop	src/core/control_loop.py	ControlLoop	Orchestrates observe → plan → validate → execute cycle
Policy Engine	src/core/policy_engine.py	PolicyEngine	Evaluates action permissions and sandbox rules
Intent Parser	src/core/intents.py	IntentParser	Converts raw input (JSON or NL) into typed Intent objects
Action Planner	src/core/action_planner.py	ActionPlanner	Converts Intents into ordered ActionStep plans
Action Executor	src/core/execution.py	ActionExecutor	Executes plan steps and captures result envelopes
System Observer	src/core/observer.py	SystemObserver	Provides environment state (idle time, processes, windows, etc.)
Logger	src/core/logger.py	AegisLogger	Structured JSONL + SQLite audit logging

3. Data Contracts

3.1 Intent Object

```
{
  "type": <string>,
  "args": <dict>,
  "confidence": <float 0-1>
}
```

Required fields: type, args

3.2 ActionStep Object

```
{
  "action": <string>,
  "params": <dict>,
  "dry_run": <bool>
}
```

3.3 ActionResult Envelope

```
{
  "result": <dict>,
  "error": <string|null>,
  "duration": <float seconds>
}
```

4. Control Loop Sequence

1. acquire_state() → SystemObserver
2. parse_intents() → IntentParser
3. plan_actions() → ActionPlanner
4. validate_plan() → PolicyEngine
5. execute_plan() → ActionExecutor
6. log_event() → AegisLogger

5. Logging Schema (JSONL)

```
{
  "timestamp": <iso8601>,
  "event": <string>,
  "intent": <string|null>,
  "payload": <dict>
}
```

6. Test Contract Baseline (v0.3)

- Total tests: 159
- Required pass rate: 100%
- Test suites:
 - test_action_planner.py
 - test_control_loop.py
 - test_cycle_mode.py
 - test_execution.py

- test_intents.py
- test_logger.py
- test_policy.py

7. Phase History

Phase	Output
A	Bootstrap CLI + legacy parser
B	Core loop + policy enforcement
3.1	Logger standardization + full test rewrite (159/159)

8. Constraints

- No implicit side effects
- All public APIs must be JSON-serializable
- All actions must route through PolicyEngine
- Logging is mandatory for every execution step

9. Future Reserved Modules

Module	Status
Desktop Automation Layer	TODO
OWUI Runtime Dashboard	TODO
Multi-Agent Router	TODO

10. UML Class Diagrams (ASCII) — v0.3

10.1 Core Orchestration

```

+-----+          uses          +-----+
validates          +-----+
| ControlLoop      |----->| ActionPlanner
|----->| PolicyEngine  |
|-----|
|-----|
| - settings      |          | - logger
|          |          | - policy
| - logger        |          |
|          |          | - system_observer
| - policy_engine |          | +plan(intents)
|          |          | +check(plan)
|          |          |

```

```

| - action_planner | +-----+
+                   +-----+
| - executor       | ^       |
| +run_n_cycles(n) | |       | produces
| +run_forever()   | |       |
| +_observe()      | +-----+
executes +-----+
| +_think(prompt)  | | IntentParser
|----->| ActionExecutor | |
| +_log(event)     | |
|-----| |-----|
+-----+ | +parse(raw)
|           | - logger |
+           +-----+
+           | +execute(plan) |
+-----+

```

10.2 Support Services

```

+-----+           writes +-----+
| SystemObserver |----->| AegisLogger |
|-----| |-----|
| +get_state()   | | +log_event(...) |
| +get_idle_time() | | +log_llm_call(...) |
+-----+ +-----+

+-----+
| OWUIClient    |
|-----|
| +chat(...)    |
+-----+

```

10.3 Data Types

```

<<value object>> Intent
- type: str
- args: dict
- confidence: float

<<value object>> ActionStep
- action: str
- params: dict
- dry_run: bool

```

```
<<envelope>> ActionResult
- result: dict
- error: Optional[str]
- duration: float
```

11. Public API Signatures (authoritative)

All functions below must remain JSON-serializable at the boundary. Arguments with complex types refer to the Data Contracts in §3.

11.1 ControlLoop (src/core/control_loop.py)

```
class ControlLoop:
    def __init__(self, *,
                 settings: dict,
                 logger: "AegisLogger",
                 policy_engine: "PolicyEngine",
                 action_planner: "ActionPlanner",
                 intent_parser: "IntentParser",
                 executor: "ActionExecutor",
                 system_observer: "SystemObserver") -> None:
        """Constructs the supervised loop. No side-effects beyond dependency
        capture."""

    def run_n_cycles(self, n: int, *, prompt: str | None = None) -> None:
        """Run exactly n observe-plan-validate-execute cycles."""

    def run_forever(self, *, sleep_seconds: float = 1.0) -> None:
        """Run indefinitely with a fixed inter-cycle sleep."""

    def _observe(self) -> dict:
        """Return current environment snapshot from SystemObserver.
        Required keys: {"timestamp", "processes", "windows", "idle_seconds"}.
        """

    def _think(self, prompt: str) -> list[dict]:
        """Optional: call a reasoning layer; returns a raw intent list (JSON-
        compatible)."""

    def _log(self, event: str, *, intent: str | None = None, payload: dict |
             None = None) -> None:
        """Structured log wrapper around AegisLogger.log_event."""
```

11.2 PolicyEngine (src/core/policy_engine.py)

```
class PolicyEngine:
    def __init__(self, *, settings: dict, logger: "AegisLogger") -> None:
        """Load policy table and path restrictions from settings."""

    def get_mode(self) -> str:
        """Return current run mode: {"assist", "auto", "sandbox"}."""

    def is_sandbox_only(self) -> bool:
        """Return True if policy limits execution to sandbox actions only."""

    def check(self, plan: list["ActionStep"]) -> dict:
        """Validate proposed plan. Return schema:
        {
            "allowed": bool,
            "requires_approval": bool,
            "denied_reason": str | None,
            "restricted_paths": list[str]
        }
        """
```

11.3 IntentParser (src/core/intent_parser.py)

```
class IntentParser:
    def __init__(self, *, logger: "AegisLogger") -> None: ...

    def parse(self, raw: str | dict | list[dict]) -> list["Intent"]:
        """Convert input into a list of typed Intent objects.
        Must accept JSON string, already-parsed dict, or list of dicts.
        """

    def parse_batch(self, raws: list[str | dict]) -> list["Intent"]:
        """Parse multiple inputs. Errors are collected and logged; invalid
        entries are dropped."""
```

11.4 ActionPlanner (src/core/action_planner.py)

```
class ActionPlanner:
    def __init__(self, *, logger: "AegisLogger", system_observer:
    "SystemObserver") -> None: ...

    def plan(self, intents: list["Intent"]) -> list["ActionStep"]:
        """Map intents to an ordered list of ActionStep objects.
```

```

        No side-effects; must be deterministic for the same inputs.
        """

    def validate_plan(self, plan: list["ActionStep"]) -> tuple[bool, list[str]]:
        """Structural validation (not policy). Returns (is_valid, errors)."""

```

11.5 ActionExecutor (src/automation/executor.py)

```

class ActionExecutor:
    def __init__(self, *, logger: "AegisLogger", dry_run: bool = False) -> None: ...

    def execute(self, plan: list["ActionStep"]) -> list["ActionResult"]:
        """Execute steps sequentially. Each step produces an ActionResult.
        Executor ensures per-step try/except and duration capture.
        """

```

11.6 SystemObserver (src/utis/system_observer.py)

```

class SystemObserver:
    def get_state(self) -> dict:
        """Return process/window/idle snapshot. Keys must include:
        {
            "timestamp": str (iso8601),
            "processes": list[dict], # name, pid, cpu, mem
            "windows": list[dict],   # title, hwnd, app
            "idle_seconds": float
        }
        """

    def get_idle_time(self) -> float:
        """Return OS idle time in seconds (float)."""

```

11.7 AegisLogger (src/aegis_logging/logger.py)

```

class AegisLogger:
    def __init__(self, *, log_dir: str, db_path: str, settings: dict | None = None) -> None:
        """Initialize JSONL + SQLite logger. Does not create sessions by default."""

    def start_session(self, *, with_title: str | None = None) -> str:
        """Open a new session; returns session_id."""

```

```

def end_session(self, session_id: str) -> None: ...

def log_event(self, event: str, *, intent: str | None = None, payload: dict | None = None) -> None:
    """Append structured event to JSONL and DB (if session active)."""

def log_llm_call(self, *, model: str, prompt: str, response: str,
                  prompt_tokens: int | None = None, completion_tokens: int | None = None) -> None:
    """Record LLM interaction with truncation safeguards."""

def get_recent(self, *, limit: int = 200) -> list[dict]: ...

```

11.8 OWUIClient (src/clients/owui_client.py)

```

class OWUIClient:
    def __init__(self, *, base_url: str, token: str, timeout: float = 30.0) -> None: ...

    def chat(self, *, model: str, messages: list[dict], temperature: float = 0.2,
              max_tokens: int | None = None, **kwargs) -> dict:
        """POST /api/chat/completions-compatible endpoint. Returns parsed JSON dict."""

```

12. UML: Detailed Class Members (ASCII)

12.1 ControlLoop

```

+ ControlLoop
- settings: dict
- logger: AegisLogger
- policy_engine: PolicyEngine
- action_planner: ActionPlanner
- intent_parser: IntentParser
- executor: ActionExecutor
- system_observer: SystemObserver
+ run_n_cycles(n: int, prompt: str|None) -> None
+ run_forever(sleep_seconds: float=1.0) -> None
# _observe() -> dict
# _think(prompt: str) -> list[dict]
# _log(event: str, intent: str|None, payload: dict|None) -> None

```

12.2 PolicyEngine

```
+ PolicyEngine
- settings: dict
- logger: AegisLogger
+ get_mode() -> str
+ is_sandbox_only() -> bool
+ check(plan: list[ActionStep]) -> dict
```

12.3 IntentParser

```
+ IntentParser
- logger: AegisLogger
+ parse(raw: str|dict|list[dict]) -> list[Intent]
+ parse_batch(raws: list[str|dict]) -> list[Intent]
```

12.4 ActionPlanner / ActionExecutor

```
+ ActionPlanner
- logger: AegisLogger
- system_observer: SystemObserver
+ plan(intents: list[Intent]) -> list[ActionStep]
+ validate_plan(plan: list[ActionStep]) -> (bool, list[str])

+ ActionExecutor
- logger: AegisLogger
- dry_run: bool
+ execute(plan: list[ActionStep]) -> list[ActionResult]
```

12.5 SystemObserver / Logger / OWUIClient

```
+ SystemObserver
+ get_state() -> dict
+ get_idle_time() -> float

+ AegisLogger
+ start_session(with_title: str|None) -> str
+ end_session(session_id: str) -> None
+ log_event(event: str, intent: str|None, payload: dict|None) -> None
+ log_llm_call(model: str, prompt: str, response: str, prompt_tokens: int|None, completion_tokens: int|None) -> None
+ get_recent(limit: int=200) -> list[dict]
```

```
+ OWUIClient
  + chat(model: str, messages: list[dict], temperature: float=0.2, max_tokens:
    int|None=None, **kwargs) -> dict
```

13. Test Matrix (overview)

Suite	Primary Coverage
test_control_loop.py	ControlLoop public methods, observe/execute flow
test_policy.py	PolicyEngine mode + permission logic
test_intents.py	IntentParser contract and error handling
test_action_planner.py	Deterministic planning, structure validation
test_execution.py	Executor envelopes, error isolation, durations
test_cycle_mode.py	Multi-cycle stability, idle-time behavior
test_logger.py	JSONL/DB persistence, LLM-call logging

14. Stability Rules

- Backward compatibility of signatures in §11 is required for minor versions ($0.x.y \rightarrow 0.x.(y+1)$).
- Private methods (prefixed with `_`) may change without notice; tests must target behavior, not private internals.
- JSON keys listed in §3 are normative; additions require minor version bump; deletions require major version bump.